

Traccia Algoritmi - Esercizi 1B e 2B

Esercizio 1 (B) - Albero Binario

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- `createTree()` che restituisce un albero vuoto;
- `insert(A, x)` che inserisce il nodo `x` nell'albero `A`;
- `left(A)` che restituisce il sottoalbero sinistro di `A`;
- `right(A)` che restituisce il sottoalbero destro di `A`;
- `info(A)` che restituisce il valore associato al nodo radice di `A`;
- `setInfo(A, x)` che modifica il valore associato alla radice di `A` impostandolo a `x`;
- `isNull(A)` restituisce `True` se l'albero `A` è vuoto e `False` altrimenti.

Due Alberi Binari A_1 e A_2 si dicono simmetrici se hanno la stessa struttura e i nodi in posizione corrispondente contengono lo stesso valore. In particolare, due alberi vuoti sono simmetrici; se uno solo dei due alberi è vuoto, essi non sono simmetrici; se entrambi sono non vuoti, essi sono simmetrici se le rispettive radici hanno lo stesso valore, i rispettivi sottoalberi sinistri sono simmetrici e i rispettivi sottoalberi destri sono simmetrici.

1. Si realizzi una efficiente funzione `isSymmetric(A)` che, dato un Albero Binario `A`, restituisca `True` se e solo se il sottoalbero sinistro e il sottoalbero destro della radice di `A` sono simmetrici. Per la realizzazione si può definire una funzione ausiliaria separata, da invocare all'interno di `isSymmetric`.
2. Si discuta, motivandola adeguatamente, la complessità temporale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca il caso migliore e il caso peggiore.
3. Si discuta, motivandola adeguatamente, la complessità spaziale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca il caso migliore e il caso peggiore.
4. Si discuta se, assumendo che `A` sia un Albero Binario di Ricerca, la complessità individuata cambierebbe, motivando la risposta.

Traccia Algoritmi - Esercizi 1B e 2B

Esercizio 2 (B) - Grafo non orientato e pesato

Siconsiderino le seguenti funzioni che implementano un grafo non orientato e pesato:

- `createGraph(m)`, che, dato un valore intero m in input, crea un grafo con m nodi identificati da numeri interi compresi in $[0, m - 1]$, senza archi;
- `insertEdge(G, x, y, w)`, che, dato un grafo G , una coppia di nodi x e y e un numero intero positivo w , aggiunge un arco tra x e y di peso w (se l'arco esiste già, ne aggiorna il peso);
- `deleteEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , rimuove, se presente, l'arco tra x e y ;
- `deleteNode(G, x)`, che, dato un grafo G e un nodo x , rimuove il nodo x da G . Se il nodo x possiede archi incidenti oppure x non è presente nel grafo G , il metodo solleva un'eccezione;
- `existsEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , restituisce `True` se esiste in G un arco tra x e y , `False` altrimenti;
- `getWeight(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y tali che esiste l'arco (x, y) , restituisce il peso di tale arco;
- `size(G)`, che, dato un grafo G , restituisce il numero di nodi del grafo;
- `nodes(G)`, che, dato un grafo G , restituisce una lista contenente gli identificatori dei nodi di G .

Si noti che, a differenza di altre tracce, l'API non fornisce una funzione `neighbors` per ottenere direttamente i vicini di un nodo: tale funzione, se necessaria, dovrà essere implementata a partire dalle primitive sopra elencate.

Una rete di magazzini è collegata mediante vie di trasporto. Il grafo G rappresenta i collegamenti disponibili tra i magazzini, mentre il peso intero positivo associato a un arco rappresenta la capacità massima di transito giornaliera del relativo collegamento, espressa in unità di merce. Si vuole determinare quante unità di merce possono essere movimentate ogni giorno da un magazzino sorgente s a un magazzino destinazione t , eventualmente attraverso magazzini intermedi, senza superare la capacità di alcun collegamento.

1. Si realizzi una funzione `neighbors(G, x)` che, dato un grafo G e un nodo x , restituisca la lista delle coppie $[y, w]$ tali che esiste l'arco (x, y) di peso w , utilizzando esclusivamente le primitive elencate sopra.
2. Si realizzi una funzione `costruisciCapacita(G)` che, dato un grafo G non orientato e pesato, restituisca una matrice `cap` di dimensione $n \times n$, con $n = \text{size}(G)$, contenente valori interi, tale che `cap[u][v]` sia uguale al peso w dell'arco (u, v) , se tale arco esiste, e sia uguale a 0 altrimenti. L'elemento `cap[u][v]` rappresenta la capacità massima di transito giornaliera tra i nodi u e v .
3. Si realizzi una funzione `movimentazioneMax(G, s, t)` che, dato un grafo G non orientato e pesato, un nodo sorgente s e un nodo destinazione t , utilizzi la funzione `costruisciCapacita` del punto precedente e restituisca il massimo numero di unità di merce che possono essere movimentate ogni giorno da s a t , rispettando la capacità massima di transito di ciascun collegamento.
4. Si analizzi e si motivi la complessità temporale nel caso peggiore di `neighbors`, `costruisciCapacita` e `movimentazioneMax`, specificando la struttura dati ausiliaria adottata.
5. Si analizzi e si motivi la complessità spaziale nel caso peggiore di `neighbors`, `costruisciCapacita` e `movimentazioneMax`.